Base R Regex Functions

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
`grepl(pat, x, perl = FALSE, fixed = FALSE, ignore.case = FALSE)`	Returns logical vector (TRUE/FALSE) indicating if pattern was matched in each string **EXAMPLE:** `grepl("^\\d+", c("123", "abc")) → c(TRUE, FALSE)`
`grep(pat, x, value = FALSE, perl = FALSE)`	Returns integer indices of elements matching pattern, or matched string values if value = TRUE **EXAMPLE:** `grep("^a", c("apple", "banana", "avocado"), value = TRUE) → c("apple", "avocado")`
`sub(pat, repl, x, perl = FALSE)`	Replaces first occurrence of pattern in each string element with replacement string **EXAMPLE:** `sub("\\d+", "X", "item 10 and 20") → "item X and 20"`
`gsub(pat, repl, x, perl = FALSE)`	Replaces all occurrences of pattern in each string element with replacement string (global) **EXAMPLE:** `gsub("\\s+", "-", "hello big world") → "hello-big-world"`
`regexpr(pat, x, perl = FALSE)`	Finds first match per element; returns integer vector of start indices with 'match.length' attribute **EXAMPLE:** `m <- regexpr("\\d+", "id: 1042"); m → 5 (match.length = 4)`
`gregexpr(pat, x, perl = FALSE)`	Finds all matches in each element; returns list of integer vectors with match offsets and lengths **EXAMPLE:** `gregexpr("\\b\\w+\\b", "foo bar baz")`
`regexec(pat, x, perl = FALSE)`	Finds first match and capturing groups; returns list of indices and lengths for each group **EXAMPLE:** `regexec("([a-z]+):([0-9]+)", "port:8080")`
`regmatches(x, m) / regmatches(x, m) <- value`	Extracts or replaces substrings from x matching positions returned by regexpr, gregexpr, or regexec **EXAMPLE:** `regmatches(x, gregexpr("\\d+", x)) → list of matched digit strings`
`strsplit(x, split, fixed = FALSE, perl = FALSE)`	Splits string vector x into substrings at pattern matches; returns list of string vectors **EXAMPLE:** `strsplit("a, b; c", "[,;]\\s*")[[1]] → c("a", "b", "c")`

stringr / Tidyverse Functions (ICU Engine)

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
`str_detect(string, pattern, negate = FALSE)`	Detects pattern matches in string vector; returns logical vector (TRUE/FALSE) **EXAMPLE:** `str_detect(c("apple", "banana"), "^a") → c(TRUE, FALSE)`
`str_which(string, pattern) / str_subset(string, pattern)`	Returns matching indices (str_which) or filtered subset of matching string values (str_subset) **EXAMPLE:** `str_subset(c("dog", "cat", "deer"), "^d") → c("dog", "deer")`
`str_count(string, pattern)`	Counts the number of non-overlapping matches in each string element **EXAMPLE:** `str_count("banana", "a") → 3`
`str_locate(string, pattern) / str_locate_all(string, pattern)`	Returns start and end character indices of first match (matrix) or all matches (list of matrices) **EXAMPLE:** `str_locate("hello 123", "\\d+") → start: 7, end: 9`
`str_extract(string, pattern) / str_extract_all(string, pattern)`	Extracts first matched substring (character vector) or all matches (list of character vectors) **EXAMPLE:** `str_extract_all("2026-08-19 and 2027-01-01", "\\d{4}-\\d{2}-\\d{2}")`
`str_match(string, pattern) / str_match_all(string, pattern)`	Extracts full matches and each capturing group into columns of a character matrix **EXAMPLE:** `str_match("user: 1042", "(\\w+): (\\d+)") → matrix with full match and group columns`
`str_replace(string, pattern, replacement)`	Replaces first match with replacement string (supports backreferences \1, \2 — NOT $1) **EXAMPLE:** `str_replace("price: $10", "\\$(\\d+)", "€\\1") → "price: €10"`
`str_replace_all(string, pattern, replacement)`	Replaces all matches; replacement can be a string or a named vector c('pat1' = 'rep1') **EXAMPLE:** `str_replace_all("a 1 b 2", c("a" = "A", "b" = "B")) → "A 1 B 2"`
`str_remove(string, pattern) / str_remove_all(string, pattern)`	Removes first or all occurrences of pattern (equivalent to str_replace with empty replacement) **EXAMPLE:** `str_remove_all("hello 123 world 456", "\\d+\\s*") → "hello world "`
`str_split(string, pattern, n = Inf, simplify = FALSE) / str_split_1(string, pattern)`	Splits strings into substrings at pattern matches; returns list, matrix (simplify=TRUE), or vector (str_split_1) **EXAMPLE:** `str_split_1("a,b,c", ",") → c("a", "b", "c")`

Flags, Modifiers & Pattern Options

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
`perl = TRUE (Base R)`	Enables PCRE (Perl-Compatible Regular Expressions) engine; enables lookarounds, non-greedy, named groups EXAMPLE: `grepl("(?<=id:)\\d+", "id:42", perl = TRUE) → TRUE`
`fixed = TRUE (Base R & stringr::fixed())`	Matches pattern as exact literal byte/character sequence; skips regex parsing for maximum speed EXAMPLE: `sub(".", "_", "a.b.c", fixed = TRUE) → "a_b.c"`
`ignore.case = TRUE (Base R)`	Performs case-insensitive matching across base R functions (grep, grepl, sub, gsub) EXAMPLE: `grepl("apple", "APPLE", ignore.case = TRUE) → TRUE`
`useBytes = TRUE (Base R)`	Performs byte-by-byte matching instead of character-by-character; faster on ASCII/raw bytes EXAMPLE: `grep("foo", text_vec, useBytes = TRUE)`
`regex(..., ignore_case = TRUE) (stringr)`	Case-insensitive modifier for stringr patterns EXAMPLE: `str_detect("HELLO", regex("hello", ignore_case = TRUE)) → TRUE`
`regex(..., multiline = TRUE) (stringr)`	Multiline mode: ^ and $ match start/end of each individual line within the string EXAMPLE: `str_extract_all(text, regex("^\\w+", multiline = TRUE))`
`regex(..., dotall = TRUE) (stringr)`	Dot-all mode: dot (.) matches any character including newline (\n) EXAMPLE: `str_extract(html, regex("<div>.*?</div>", dotall = TRUE))`
`regex(..., comments = TRUE) (stringr)`	Verbose mode: ignores whitespace and enables inline comments starting with # inside pattern EXAMPLE: `regex("\\d{4} # year", comments = TRUE)`
`coll(pattern, locale = 'en') (stringr)`	Locale-aware collation matching according to standard international sorting rules EXAMPLE: `str_detect("resume", coll("résumé", ignore_case = TRUE))`

R-Specific Syntax, POSIX Classes & Escapes

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
`"\\d", "\\w", "\\s", "\\b"`	Double backslashes in standard R strings: R string parser requires escaping the backslash itself EXAMPLE: `grepl("\\d+", "123") vs r"(\d+)"`
`r"(...)", r"-(...)-", r"{\d+}"`	Raw string literals (R 4.0.0+): No backslash escaping required; delimiters (), [], {}, or -()- EXAMPLE: `r"(\b[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}\b)"`
`[[:digit:]], [[:alpha:]]`	POSIX character classes: Digits [0-9] and Alphabetic characters [a-zA-Z] EXAMPLE: `gsub("[[:digit:]]", "#", "a1b2") → "a#b#"`
`[[:alnum:]], [[:punct:]]`	POSIX character classes: Alphanumeric characters and Punctuation marks EXAMPLE: `gsub("[[:punct:]]", "", "Hello, World!") → "Hello World"`
`[[:space:]], [[:blank:]]`	POSIX character classes: Whitespace [\t\r\n\v\f] and Blank characters (space/ tab only) EXAMPLE: `strsplit(text, "[[:space:]]+")[[1]]`
`[[:lower:]], [[:upper:]]`	POSIX character classes: Lowercase letters [a-z] and Uppercase letters [A-Z] EXAMPLE: `grep("^[[:upper:]]", c("John", "jane"), value = TRUE) → "John"`
`[[:xdigit:]], [[:cntrl:]]`	POSIX character classes: Hexadecimal digits [0-9a-fA-F] and Control characters EXAMPLE: `grepl("^[[:xdigit:]]+$", "1A4F") → TRUE`
`(?<name>...)`	Named capture group in PCRE (perl = TRUE) and stringr; accessible by column name or group name EXAMPLE: `str_match("2026-08-19", "(?<y>\\d{4})-(?<m>\\d{2})-(?<d>\\d{2})")`
`\\1, \\2`	Numbered backreferences in replacement strings (sub, gsub, and stringr all use \1, \2 — R engines do not support $1) EXAMPLE: `gsub("(\\w+)\\s+(\\w+)", "\\2, \\1", "John Doe")`
`(?<=prefix), (?<!prefix)`	Positive and negative lookbehinds (requires perl = TRUE in Base R or stringr) EXAMPLE: `str_extract("price: $42", "(?<=\\$)(\\d+)") → "42"`